CS 498 - Homework 3

Bigtable Schema Design and Performance Analysis

Part 1: Wide-Column Schema Design

1. Row Key and Sharding

I would use hash(DOL Vehicle ID)#DOL Vehicle ID as the row key. The hash prefix helps spread records across Bigtable tablets and reduces hotspotting. However, searching by City or Make would be less efficient without a separate lookup table.

2. Column Families

I would use three column families:

- **vehicle:** VIN, Make, Model, Model Year
- **location:** City, County, State, Postal Code
- **details:** Electric Range, Base MSRP, Eligibility

3. Horizontal Scaling

A hashed shard key distributes writes more evenly and reduces hotspots, but makes range queries harder. A range-based key using Model Year makes year-based queries easier, but newer years could receive more traffic and create hotspots.

Part 4: Performance Analysis

1. Average Latency

I tested 500 requests for each endpoint using a Python script. The average response times from the second run were:

- **insert-fast:** 30.15 ms
- **insert-safe:** 31.74 ms

2. Results

The primary only writes were slightly faster in the second run. This is likely because w=1 only waits for the primary to acknowledge the write, while w="majority" waits for more replicas. The first run had a much higher fast-write average of 506.12 ms, showing that other conditions can also affect latency.

3. CAP Theorem Analysis

I would choose primary only writes for something like social media likes, where losing a few recent updates would not be as serious. Using w=1 can reduce write latency, but there is a higher risk of losing recent data if the primary fails before replication. It also does not guarantee availability during a network partition.